Technical Report - Data Visualisation Final Project

Student: Abdulrahman Shaalan

Student No: 210201916

Executive Summary

This project contains two visual analytics components. The first component is a social network visualization that examines relationships between fictional characters using a weighted interaction graph. The goal is to reveal communities, identify important characters, and explain how network structure supports interpretation. The second component is a real-time Bitcoin monitoring dashboard that uses a live API feed, temporal line chart, and conditional alerts to help users detect price movement quickly.

Track A: Social Network Visualization & Community Discovery

Dataset and Relationship Definition

The network represents character interactions. Each node represents a character, and each edge represents a relationship or meaningful interaction between two characters. The edge weight represents the strength of the relationship. A larger weight indicates a stronger or more frequent connection, while a smaller weight indicates a weaker relationship.

The project includes two formal data tables:

- nodes.csv: contains each character and a general group hint.
- edges.csv: contains source, target, and relationship weight.

Community Detection

The Louvain algorithm was used to partition the graph into communities. This method is appropriate because it attempts to group nodes that are densely connected with each other while separating them from nodes with weaker external connections. In the visualization, detected communities are shown using color groups.

Visual Centrality

Node size is mapped to degree centrality. This means that characters with more direct relationships appear larger in the graph. This visual hierarchy makes the network more glanceable because important actors can be identified without reading every label.

Layout Justification

A force-directed layout was selected because it naturally pulls strongly connected nodes closer together and pushes weakly connected groups apart. This makes community structure easier to see. Bridge nodes also become more visible because they tend to appear between clusters.

Community Narrative

The detected communities generally reflect recognizable story groups, such as Stark-related characters, Lannister-related characters, and Targaryen-related supporters. The separation between these groups shows how the network is organized around political and family alliances. Characters such as Tyrion Lannister and Jon Snow are expected to appear as important nodes because they connect multiple parts of the story network.

Weak Link Analysis

A bridge node or edge is important because it connects two otherwise separated communities. For example, a relationship between Jon Snow and Daenerys Targaryen can act as a bridge between the Stark/Northern side and the Targaryen alliance. If this bridge were removed, the network would likely become more separated, making communication between those clusters less direct.

Visualization Critique

One limitation is that some edges may overlap when the graph becomes larger. If more time were available, filtering controls could be added to hide weak ties or show only the most important nodes. This would reduce visual noise and improve focus.

Track B: Real-Time Data Visualization

Data Pipeline Architecture

The dashboard uses the CoinGecko API as the live data source. The app sends REST requests at a selected refresh interval. This is a polling approach rather than a WebSocket push approach. Polling is easier to implement, stable for a student project, and suitable because the dashboard only needs periodic updates rather than extremely high-frequency streaming.

Polling Frequency

The dashboard uses a configurable refresh rate between 10 and 60 seconds. This balances the need for current information with responsible API usage. Refreshing too quickly may waste calls because public API data is often cached or rate-limited.

Temporal Window

The dashboard keeps recent observations in a sliding window. The main line chart shows recent price movement rather than all historical data. This keeps the visualization focused on the current situation and supports quick trend detection.

Alerting and Threshold Logic

The dashboard calculates the percentage change across the sliding window. If the absolute price change exceeds the selected threshold, the interface displays a red alert. If the value is within the threshold, the dashboard displays a normal status message. This satisfies the requirement for conditional formatting and supports fast anomaly detection.

Visual Encoding

The dashboard uses metrics for current state, a line chart for temporal movement, and red alert formatting only for important changes. This reduces cognitive load because bright colors are reserved for meaningful events. The dashboard also displays connection status and last update time to help users notice latency or API problems.

Performance and Latency

The dashboard uses lightweight REST requests and stores only the latest observations. This avoids unnecessary memory growth. The main possible delay is API response time or cached data. The connection status helps users understand whether the dashboard is receiving new data successfully.

Future Scaling

If the data source suddenly produced 1,000 updates per second, the current Streamlit design would not be enough. The first issue would be frontend rendering and session-state storage. A scalable solution would use a message queue or streaming service, aggregate values before visualization, and update the chart at a lower visual refresh rate.

Conclusion

The final project demonstrates both structural and temporal visualization. Track A focuses on network structure, community detection, centrality, and interpretive analysis. Track B focuses on live data, temporal context, alerting, and dashboard usability. Together, the two parts satisfy the core technical, design, and analytical requirements of the final project.